



Program: BS Computer Science

Semester: Fall 2025

Credit Hour(s): 1

Pre-requisite(s): EE1005 Digital Logic Design

Instructor: Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

Lab # 5: Branching, Flags, Comparison and Conditions

1. Objectives

- Understand how the **CMP instruction** affects flags.
- Learn the role of different **flags** in decision making.
- Apply **conditional jumps** to control program flow.

2. Branching

Branching means **changing the normal flow of program execution**. Normally, instructions run *sequentially* (line by line, top to bottom). But with branching, the program can:

- Skip instructions
- Jump to another location
- Repeat a block of code (loops)
- Make decisions (if / else)

Types of Branching

1. **Conditional Branching:** The program jumps only if a *certain condition is true*, based on flags.
2. **Unconditional Branching:** The program *always jumps* to a new instruction.

3. CMP Instruction

The CMP (compare) instruction is fundamental to branching. It compares *two operands* by internally performing a subtraction but *without storing the result*. Instead, CMP only purpose is to update the flags.

Syntax: `CMP destination, source`

Example:

- `CMP AX, BX`
- `CMP AL, 10`
- `CMP [NUM], DX`



4. Flags

The Flags Register is a special register that contains a collection of single-bit flags. Each flag is set (1) or cleared (0) by an instruction to indicate the status of an operation.

Flag	Flag Name	Meaning
ZF	Zero Flag	Set if result = 0
SF	Sign Flag	Set if result is negative
CF	Carry Flag	Set if borrow needed (unsigned)
OF	Overflow Flag	Set if signed overflow occurs
PF	Parity Flag	Set if number of 1 bits in result is even
AF	Auxiliary Carry	Set if carry/borrow occurs from bit 3 to bit 4 (used in BCD)

5. Examples of Flags

5.1. Zero Flag (ZF): The Zero Flag is set (ZF=1) if the result of an operation is zero. It is cleared (ZF=0) otherwise. It is most commonly used with the CMP instruction to check for equality.

```
MOV AX, 5
CMP AX, 5      ; 5 - 5 = 0 → ZF = 1
```

```
MOV AX, 7
CMP AX, 9      ; 7 - 9 = -2 → ZF = 0
```

```
MOV AX, 7
MOV BX, 7
CMP AX, BX     ; 7 - 7 = 0 → ZF = 1
```

```
MOV AX, 5
ADD AX, 3      ; 5 + 3 = 8 → ZF = 0
```

```
MOV AX, 0
AND AX, 0FFh   ; 0 & 255 = 0 → ZF = 1
```

5.2. Sign Flag (SF): The Sign Flag reflects the sign of the result of an operation. It is set (SF=1) if the result is negative (its most significant bit, or MSB is 1) and cleared (SF=0) if the result is positive or zero.

```
MOV AX, 5
MOV BX, 10
CMP AX, BX     ; 5 - 10 = -5 (negative) → SF = 1
```



```
MOV AX, 12
CMP AX, 4      ; 12 - 4 = +8 → SF = 0
```

```
MOV AX, 0
SUB AX, 0      ; 0 - 0 = 0 (not negative) → SF = 0
```

5.3. Carry Flag (CF): The Carry Flag is set (CF=1) if an arithmetic operation generates a carry out of the most significant bit for addition, or a borrow in for subtraction. It is essential for unsigned arithmetic.

```
MOV AL, 5
CMP AL, 7      ; 5 - 7 → borrow → CF = 1
```

```
MOV AL, 10
CMP AL, 3      ; 10 - 3 → no borrow → CF = 0
```

```
MOV AL, 0
CMP AL, 1      ; 0 - 1 → borrow → CF = 1
```

```
MOV AL, 255
ADD AL, 1      ; 255 + 1 = 256 (carry) → CF = 1
```

```
MOV AL, 0
SUB AL, 1      ; 0 - 1 = -1 (borrow) → CF = 1
```

```
MOV AL, 100
ADD AL, 50     ; 100 + 50 = 150 (no carry) → CF = 0
```

5.4. Overflow Flag (OF): The Overflow Flag is set (OF=1) if an operation results in a signed overflow. This occurs when the result is too large to fit in the destination register signed range.

```
MOV AL, 127
ADD AL, 1
; 127 + 1 = 128 (becomes -128 in signed, overflow) → OF = 1
```

```
MOV AL, 50
ADD AL, 20     ; 50 + 20 = 70 (no overflow) → OF = 0
```

```
MOV AL, -128
SUB AL, 1      ; -128 - 1 = -129 (signed overflow) → OF = 1
```

```
MOV AL, -60
ADD AL, -60    ; -60 + (-60) = -120 (within range) → OF = 0
```



5.5. Parity Flag (PF): The Parity Flag is set (PF=1) if the lower 8 bits of the result of an operation contain an *even number of set bits (1s)*. It is used for data integrity checks.

```
MOV AL, 2      ; 00000010 → 1 one → odd
TEST AL, AL    ; updates flags based on AL
; PF = 0
```

```
MOV AL, 3      ; 00000011 → 2 ones → even
TEST AL, AL
; PF = 1
```

```
MOV AL, 7      ; 00000111 → 3 ones → odd
TEST AL, AL
; PF = 0
```

```
MOV AL, 0      ; 00000000 → 0 ones → even (zero)
TEST AL, AL
; PF = 1
```

5.6. Auxiliary Carry (AF): The Auxiliary Carry Flag is set (AF=1) if an arithmetic operation causes a carry or borrow from the lower nibble (bits 0-3) to the higher nibble (bits 4-7). It is mainly used in BCD (Binary-Coded Decimal) arithmetic.

Carry from bit 3 → bit 4

```
MOV AL, 9
ADD AL, 1      ; 9 + 1 = 10 (no carry from low nibble) → AF = 0
```

```
MOV AL, 10h   ; 00010000
SUB AL, 1     ; Borrow from high nibble → AF = 1
```

```
MOV AL, 5
ADD AL, 3     ; 5 + 3 = 8 (no carry between nibbles) → AF = 0
```

```
MOV AL, 0x0F
ADD AL, 1     ; 0x0F + 1 = 0x10 → AF = 1
```

```
MOV AL, 0x08
ADD AL, 1     ; 0x08 + 1 = 0x09 → AF = 0
```

```
MOV AL, 0x19
ADD AL, 7     ; 0x19 + 7 = 0x20 → AF = 1
```



6. Conditional Jump Instructions

Conditional jumps check flags after CMP (or arithmetic instructions).

Instruction	Meaning	Condition (after CMP)
JE / JZ	Jump if equal / zero	ZF = 1
JNE / JNZ	Jump if not equal / not zero	ZF = 0
JG / JNLE	Jump if greater (signed)	ZF=0, SF=OF
JL / JNGE	Jump if less (signed)	SF ≠ OF
JA	Jump if above (unsigned greater)	CF=0, ZF=0
JB	Jump if below (unsigned less)	CF=1
JGE	Jump if greater or equal (signed)	SF=OF
JLE	Jump if less or equal (signed)	ZF=1 or SF≠OF
JO	Jump if Overflow	OF=1

7. Example Programs

Example 1:

```
[org 0x0100]
    mov ax, 0
    mov bx, 0
    mov cx, 10
loop_start:
    add ax, [bx + data]
    add bx, 2
    dec cx
    jnz loop_start
mov [result], ax

mov ax, 0x4c00
int 0x21

data: dw 10, 20, 30, 40, 50, 60, 70, 80, 90, 99
result: dw 0
```

Example 2:

```
[org 0x0100]
    mov ax, 0
    mov bx, 0
loop_start:
    add ax, [bx + data]
    add bx, 2
```



```
    cmp bx, 20
    jne loop_start

mov  [result], ax

mov  ax, 0x4c00
int  0x21

data:  dw  10, 20, 30, 40, 50, 60, 70, 80, 90, 99
result: dw  0
```

Example 3:

```
[org 0x0100]
    mov bx, 0
    mov ax, [data + bx]
    add bx, 2
    mov cx, 9
loop_start:
    cmp ax, [data + bx]
    jge skip_update
    mov ax, [data + bx]
skip_update:
    add bx, 2
    dec cx
    jnz loop_start
mov  [result], ax

mov  ax, 0x4c00
int  0x21

data:  dw  10, 20, 30, 40, 50, 60, 70, 80, 90, 99
result: dw  0
```

8. Lab Tasks

Task 1: Sum of Array (Loop with Counter)

Write a program that:

1. Initializes an array of 7 numbers.
2. Uses **CX** as a loop counter.
3. Sums all numbers and stores the result in memory.

(Hint: Use **jnz** to control the loop)



Task 2: Sum of Array (Branch with CMP)

Modify the program in *Task 1* to:

1. Remove the use of CX as counter.
2. Instead, use **CMP BX, 20** and **JNE** to stop when BX reaches the end of the array.

(Hint: Each word is 2 bytes → 10 elements = 20 bytes total)

Task 3: Find the Maximum Number

Write a program that:

1. Assumes the **first element** of the array is the maximum.
2. Compares it with the next element using **CMP**.
3. If the new element is larger, update AX.
4. At the end, store the largest number in memory.

(Hint: Use **CMP AX, [data+bx]** and a conditional jump like **JGE** or **JL**.)

Task 4: Flags Demonstration

Write short code snippets to demonstrate how each of the following flags changes.

For each snippet, record the values of the flag in the debugger.

1. **Zero Flag (ZF)**: Show when subtraction results in zero, non-zero.
2. **Sign Flag (SF)**: Compare two numbers where result is negative and positive.
3. **Carry Flag (CF)**: Compare values where borrow is required.
4. **Overflow Flag (OF)**: Add signed values that cause overflow.
5. **Parity Flag (PF)**: Load values into AL and observe PF when binary representation has even/odd number of 1 bits.
6. **Auxiliary Carry Flag (AF)**: Add small BCD values (like 0x0F + 1) to trigger AF.

Task 5: Classify numbers as Zero or Non-Zero, and then determine further properties.

1. Load a number from an array
2. Do `cmp ax, 0` → now:
 - If equal (ZF=1) → number is zero, store 0
 - If not equal (JNZ) → number is non-zero, continue
3. For non-zero numbers:
 - If negative → observe **SF=1** in debugger
 - If subtraction caused borrow → observe **CF=1**
 - If addition later causes overflow → observe **OF=1**
 - If even/odd → use **TEST** and check **PF**
 - If nibble carry → show **AF**
4. Store classification into a result array